

Icarus 智测 · 功能总览

面向读者：想快速了解这个项目现在能做什么的人（评委、同学、潜在使用者）。
本文只讲当前真实可用的功能与实测结果，每一节都给出可复制的命令；全部数字都由脚本现算，不手写。技术细节与实现原理见各功能对应的专门文档。

更新时间：2026-09-11

一、一句话与五分钟上手

AI 提出测试假设，开源仿真器作出判决。
把自然语言的验证目标，变成受严格约束的测试计划，再生成确定性的 testbench，交给 Icarus Verilog 真实编译与仿真，最后产出可追溯的证据与报告。AI 不参与判分。

```
# 0. 准备 (只需 Python 3.11+ 与 Icarus Verilog)
python -m pip install -e .
python -m pip install pytest # 跑测试用
python -m pip install yowasp-yosys # 可选：综合证据层

# 1. 三十秒确认工具都在位
python -c "from iverilog_ai.core.toolchain import locate_tools, describe_tools;
print(describe_tools(locate_tools()))"

# 2. 跑一个真实案例：AI 出计划 → 生成 testbench → Icarus 裁决 → 报告
python -m iverilog_ai plan-run \
--plan examples/simple_alu_plan.json \
--contract examples/simple_alu_contract.json \
--rtl rtl/simple_alu.v \
--iverilog D:\iverilog\bin\iverilog.exe \
--vvp D:\iverilog\bin\vvp.exe

# 3. 一键复现基准结论 (14 参考设计 + 80 缺陷变体)
python scripts/run_benchmark_matrix.py

# 4. 打开网页演示 (导入自己的 RTL、看波形结论、跑行为对比)
streamlit run ui/app.py
```

第 2 步不需要任何 API 密钥、不需要联网 —— 默认走离线 MockProvider，或另开终端 python -m iverilog_ai.ai.debug_server 走本地调试模型。

二、当前规模 (由脚本现算)

项目	数量	现算方式
基准案例	14	benchmarks/manifest.json 的 categories
可复现缺陷变体	80	同上 defects
参考 RTL 文件	95 (15 参考 + 80 缺陷)	rtl/.v

手写 testbench	25	tb/.v
静态检查规则	44 (error 4 / warn 27 / info 13)	规则注册表现算
已对齐参考模型 (权威预言机)	14 / 14	SUPPORTED == set(AUTHORITATIVE)
自动化测试	234	python -m pytest -q
上游实测约定来源	2 个开源项目、172 个文件	data/opensource_conventions.json

基准矩阵最近一次实跑结论：

指标	结果
参考设计通过	14 / 14
缺陷检出	80 / 80 (100%)
参考误报	0
不可判定	0

三、功能总表

#	功能	一句话说明	入口
1	AI 测试计划与受控执行	自然语言 → 受 Schema 约束的 TestPlan → 确定性 testbench → Icarus 裁决	CLI plan-run、网页
2	权威预言机	期望值由与 RTL 逐拍对齐的参考模型独立复算并覆盖 AI 数字	流水线自动
3	波形语义分析	边沿统计、毛刺判定、晚/早一拍相位、失败时间窗、双波形差异	报告、网页
4	静态质量审查	44 条规则 + 质量评分，每条配正反例	review_rtl_file、网页
5	分层证据 (含综合)	仿真/综合/时序/比特流/上板五层状态显式列出，未做的标"未运行"	CLI --synth、网页
6	行为级对比	两份 RTL 跑同一份 TestPlan，逐检查项 + 逐波形信号比对	CLI compare-rtl、网页
7	开源规约知识摄取	从真实开源项目实测编码约定，注入 AI 规划上下文	scripts/ingest_open_source_conventions.py

8	结构化断言	受限模板（相等/稳定/永不置高），拒绝自由 Verilog 代码	网页、TestPlan
9	离线调试接口	无密钥、不联网跑通完整 AI 路径	<code>python -m iverilog_ai.ai.debug_server</code>

四、逐项功能说明

功能 1：AI 测试计划与受控执行

它做什么：把"覆盖复位、状态转换与边界时序"这类自然语言目标，变成结构化的 TestPlan（每个向量的输入、周期数、期望值），再由确定性模板生成 Verilog-2001 testbench。生成器不解析、不拼接模型给的任何文本，只把已校验的标量值填进固定模板。

四重门控（这是本项目与"让大模型直接写 testbench"的根本区别）：

门	机制	拦截什么
一	严格 JSON Schema 校验	字段名错误、杜撰字段、越界取值
二	DUT 合约匹配	模型杜撰端口别名（把 enable 写成 en）
三	参考模型复算并覆盖 AI 期望值	AI 写错期望值导致假通过/假失败
四	结构化断言只用模板	模型注入自由 Verilog/SVA 代码

实测：在线真实模型实验（11 案例、256 次仿真）四个策略的请求级计划合法性均为 100%、参考误报（硬失败）均为 0。离线 MockProvider（默认路径）与本地调试模型都能在不联网、不用密钥的前提下走通全流程。后续的双模型对比实验覆盖 12 个案例 / 62 个缺陷（比基准集少 16 个缺陷，因为纯组合逻辑案例无法用向量式 testbench 表达时序语义，运行器会显式跳过并给出原因，而不是静默漏算）。

- 口径说明：两份实验的缺陷总数不同（54 / 62），因为案例集合随基准集扩展而变化；
- 同一张表内的比较始终是同一批案例，跨表比较请注意分母。
- 另一处口径：早期实验里记录了 2 处"参考设计期望值不一致"，那是 AI 猜错期望值的
- 诊断指标（该案例当时尚未纳入权威预言机），不计入参考误报。参考模型全覆盖之后，运行器里的该项统计已归零。

边界：模型从不接触 Verilog 语法层，因此结构上不可能注入代码、命令或路径。

```
python -m iverilog_ai plan-run --plan <plan.json> --contract <contract.json> --rtl <dut.v> ...
```

功能 2：权威预言机（让 AI 无法"制造"结论）

它做什么：内置案例的期望值由 Python 参考模型独立复算，覆盖 AI 给出的数字。只有与 RTL 逐拍对齐的模型才能进入 AUTHORITATIVE 集合；未对齐的设计返回空字典，调用方回退为 AI 期望值并如实标注 `expectation_source="ai_generated"`。

为什么必须这样：模型一旦与 RTL 差一拍，预言机就会把"参考设计通过"改写成"参考设计失败"——比没有预言机更糟。因此规则是"对齐一个、加入一个"。

实测：14/14 个内置案例在 4 个确定性随机种子上逐拍零差异。

对齐过程顺带发现的真问题：8 个模型错误，其中 3 个同源于 IEEE 1364 非阻塞赋值语义 (`bit_idx <= bit_idx + 1` 之后再读 `frame[bit_idx+1]`，读到的是旧 `bit_idx`)。详见 [docs/reference_model_alignment.md](#)。

功能 3：波形语义分析

它做什么：直接读 VCD（不依赖 GTKWave），给出可核对的波形事实：

边沿识别：每个信号的 `rise / fall / initial` 序列（x/z 会清空上一个已知值）

毛刺判定：全局基线 + 连续 ≥ 3 个异常间隔 + 复位沿不参与统计

相位检查：输出晚一拍/早一拍，取最小延迟（不是第一个响应沿）

失败时间窗：把失败周期映射回 VCD 时间片段

双波形差异：参考 RTL 与缺陷 RTL 的逐信号差异（跳变数/时刻/数值序列）

实测（真实缺陷）：`uart_tx` 位序错误 → 6 处差异；`spi_master` 多发一位 → 20 处；`pwm` 极性反转 → 5 处，`dut_i.pwm_out` 参考 12 次跳变、缺陷 13 次。

判据经三轮校准才做到不误报：第一版"窗口内翻转多次即不稳定"在时钟与计数器上全面误报；第二版"短于全局中位数 1/3"仍把 `busy` 信号的正常窄脉冲报成毛刺；第三版才同时容下均匀节奏、脉冲、双峰节奏。校准记录见 [docs/vcd_analysis.md](#)。

边界：只分析 VCD 中 `dump` 出来的信号；结论不参与 PASS/FAIL 裁决。

功能 4：静态质量审查

它做什么：对 RTL 源码跑 44 条规则，输出命中位置、质量评分、`error/warn/info` 分级与修复建议，可导出 Markdown/JSON。

一个必须理解的边界（实测对照）：

文件	命中	质量评分
<code>rtl/mod10_counter.v</code> （参考）	1 条 <code>warn</code>	95
<code>rtl/mod10_counter_bug_no_wrap.v</code> （回绕缺陷）	1 条 <code>warn</code>	95

两者评分完全一样。这说明功能缺陷通常不是风格问题，静态审查替代不了仿真——这正是本项目把"判决权交给 Icarus"的原因之一。

噪声校准：初版 9 条规则对仓库内 84 个 RTL 文件报出 317 条命中，含大量误报（`division-operator` 把注释结尾 / 当除法，84/84 全命中；`inferred-latch` 把合法写法判成锁存器，15 条 `error` 级误报）。校准后：44 条规则、163 条命中、0 条 `error` 级误报。

每条规则都配最小反例与最小正例，并有断言强制"注册表 ↔ 用例表——对应"。

功能 5：分层证据（含综合）

它做什么：用一张表把证据分层，并把没做的事也写出来：

层级	状态
功能仿真	本次流水线提供 (Icarus 编译 + vvp + 结构化断言)
逻辑综合	已提供 (可选, Yosys 通用门级映射 + 单元统计)
时序分析	未运行 (需要目标器件时序库与时钟约束)
布局布线/比特流	未运行 (需要厂商工具链)
上板验证	未运行 (需要实际硬件)

实测：94 个 RTL 变体 (14 参考 + 80 缺陷) 全部可综合。14 个参考设计的通用门级单元数：sync_fifo 34、spi_master 33、uart_tx 29、simple_alu 20、traffic_light_emergency 12、debounce 9、handshake_stage 5、pwm 5、edge_detector 4、mod10_counter 4、mux4 4、sequence_101_overlap 3、johnson_counter 2、sync_reset 2。

负例验证：变量上界的 while 循环被 Yosys 明确拒绝，综合层报失败 —— 这一层不是装饰。另记录一个实测反直觉结论：#5 q <= d; 这类延时会被 Yosys 静默忽略，所以“综合通过”也不能反过来当作时序语义正确的证据。

```
python -m iverilog_ai plan-run ... --synth # 附加综合证据层
```

功能 6：行为级对比

它做什么：让你的 RTL 与参考 RTL 跑同一份 TestPlan (同一份合约、逐字节 相同的 testbench)，然后比两件事：逐检查项结果、逐波形信号。

对比对象	判定	关键证据
与自己比	identical	28/28 检查项一致，波形 0 差异
极性反转缺陷	different	27 处检查项差异；pwm_out 跳变数 12 vs 11
计数差一缺陷	different	6 处检查项差异；跳变数 12 vs 4
语义等价重写 (多出内部信号 next_count)	identical	多出的信号被识别为实现细节，不计入差异

最后一行是关键：对比看的是行为，不是文本，也不是内部信号数量。

```
python -m iverilog_ai compare-rtl `
--plan <plan.json> --contract <contract.json> `
--user-rtl <你的.v> --reference-rtl rtl/pwm.v
# 退出码：identical=0 / different=1 / 无法判定=2
```

功能 7：开源规约知识摄取

它做什么：从真实开源 Verilog 项目度量编码与验证约定，产出带出处、许可证与 每文件 sha256 的机器可读规约包，并注入 AI 规划上下文。

当前来源：alexforenych/verilog-axi (MIT, 83 文件)、ZipCPU/wb2axip (GPL-3.0-or-later, 89 文件)，都按固定提交拉取。

实测约定（两个项目的对比本身就说明"约定是项目相关的"）：

探针	verilog-axi	wb2axip
文件声明 timescale	100% (83/83)	7% (6/89)
模块使用 ANSI 端口列表	100% (55/55)	96% (86/90)
时序块没有阻塞赋值（for 变量除外）	96% (120/125)	95% (1427/1505)
case 带 default 分支	0% (0/35)	78% (114/146)
低有效复位命名为 xxx_n	0% (0/147)	24% (54/228)

口径：只记录聚合统计与文件指纹，不复制上游任何代码，因此不构成衍生作品、不触发再分发义务。覆盖率不足的探针标 weak，提示词里明确写成"上游较少如此，不构成约定"。

它的作用范围：只影响模型怎么写测试计划，不改变判定口径，也不能作为期望值依据。
IVERILOG_AI_CONVENTIONS=0 可关闭，便于做 A/B 对照。

```
python scripts/ingest_open_source_conventions.py # 联网度量
python scripts/ingest_open_source_conventions.py --offline # 复用缓存，完全离线
```

功能 8：结构化断言

它做什么：允许声明有限几种断言模板（信号相等、信号稳定、永不置高），由生成器翻译成 testbench 检查。不接受自由书写的 Verilog 或 SVA 代码，因此模型无法借断言注入代码。

边界：没有可观察样本时报告 no observable samples，不会伪造通过。

功能 9：离线调试接口

它做什么：python -m iverilog_ai.ai.debug_server 起一个 OpenAI 兼容的本地服务，只监听回环地址、不联网、不需要密钥，让"在线代码路径"可以被离线回归与演示。

用途：评委/同学在没有 API 凭据的环境下也能完整跑通流程与实验。
它不代表任何真实模型能力，其数据不作为 AI 效果依据。

五、项目边界（必须知道，也是设计的一部分）

边界	说明
仿真通过 ≠ 可综合	两者是独立属性，项目分成两层证据分别报告
综合通过 ≠ 时序收敛	综合用通用单元库，没有时序模型，不给出频率结论
综合通过 ≠ 能上板	没有引脚/IO/时钟约束与布局布线，不存在"已实现"的证据
不做代码覆盖率	报告里的"覆盖"是测试计划执行覆盖率，不是 RTL 代码覆盖率
不做形式化验证	静态规则是保守的 lint 式检查，不做完整语法树解析

静态审查替代不了仿真	实测：参考设计与功能缺陷的静态评分完全相同（95 分）
参考模型只覆盖内置案例	自定义设计的期望值来源只能是 AI 或人工，工具会显式标注来源
自定义 RTL 不是安全沙箱	请勿用来路不明的设计
波形分析只看 dump 的信号	\$dumpvars 之外的内部层次不可见

六、入口速查

命令行 (python -m iverilog_ai <子命令>)

子命令	用途
run / simulate	跑指定 RTL + testbench，执行 vvp，产出结构化结果与报告
plan-run / pipeline	校验 AI 计划与合约 → 生成 testbench → Icarus/vvp → 报告（可加 --synth）
compare-rtl	行为级对比：同一份 TestPlan 跑两份 RTL
report	从 result.json 重新渲染 Markdown / HTML
validate-plan / plan	只校验测试计划 JSON

网页 (streamlit run ui/app.py)

按顺序可看到：实证状态面板 → 案例选择 / 自定义 RTL 导入 → DUT 合约编辑与校验 → 结构化断言 → AI 接口设置 → 生成并查看 TestPlan → 执行流水线 → 结论与期望值来源 → 分层证据 → 测试覆盖摘要 → 失败反例与解释 → 波形（含语义结论、相位检查、失败时间窗） → 静态质量审查 → 结构对比与行为级对比。

主要脚本

脚本	用途
scripts/run_benchmark_matrix.py	14 参考 + 80 缺陷的固定矩阵实跑
scripts/run_strategy_experiment.py	固定/随机/离线AI/在线AI 四策略公平对比
scripts/compare_models.py	多个在线模型横向对比（含可比性检查）
scripts/ingest_open_source_conventions.py	从开源项目度量约定
scripts/summarize_trial_feedback.py	校验并汇总本地试用反馈
scripts/make_report_figures.py	生成报告插图（确定性、无第三方素材）
scripts/markdown_to_pdf.py	Markdown → PDF（离线渲染）
scripts/strip_bom.py	BOM 卫生检查（CI 门禁）

想动手试的人看这里

docs/trial/ 是一套可执行的试用材料：三个任务（基准矩阵 / AI 流程 / 自定义 RTL），约 40 分钟，全程不需要 API 密钥，附排查表与结构化反馈表。跟着跑完可以顺便帮我们发现文档缺陷——卡住的位置就是文档缺口。当前还没有真人试用记录，结果页（docs/trial/results.md）保持"待收集"状态。

七、怎么验证我不是在自说自话

```
# 1. 全量测试（含多个真实跑 Icarus 的端到端用例）
python -m pytest -q # 期望 234 passed

# 2. 基准矩阵（固定向量，结果确定）
python scripts/run_benchmark_matrix.py # 期望 14/14、80/80、0 误报、0 不可判定

# 3. 参考模型与 RTL 逐拍对齐（4 个随机种子，零差异）
python -m pytest tests/core/test_reference_model_alignment.py -q

# 4. 静态规则每条都有正反例
python -m pytest tests/core/test_static_review_rules.py -q

# 5. 工具探测（换台机器先跑这个）
python -c "from iverilog_ai.core.toolchain import locate_tools, describe_tools;
print(describe_tools(locate_tools()))"
```

CI (.github/workflows/ci.yml) 会在 Linux 与 Windows × Python 3.11/3.12 上跑同样的命令。工具位置由 core.toolchain 统一探测（环境变量 → PATH → 常见目录），因此 CI 里是真正执行仿真，而不是把测试 skip 掉。

八、知道了这些之后，建议先玩哪三个

看判决是怎么来的：跑第 3 步的基准矩阵，然后打开

.iverilog-ai/benchmark-matrix/matrix.md —— 78 个缺陷的检出记录一目了然。

看 AI 到底做了什么、没做什么：起离线调试模型 + 网页，走一遍

"生成计划 → 执行 → 报告"，重点看结果区的期望值来源提示与分层证据表。

拿自己的 RTL 试：网页切到"自定义 RTL"，上传后先看结构对比，再用

compare-rtl 或网页按钮跑行为级对比，看它怎么区分"实现细节"与"行为差异"。